



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space: Represents the set of all abstract physical configurations/states reachable in the environment, typically represented as a graph where nodes are unique world states and edges are possible actions.

Search Tree: An explicit, dynamic tree structure constructed during the search process. Its nodes represent paths through the state space. Because multiple paths can lead to the same state, a search tree can contain redundant or infinite nodes corresponding to a single physical state in the state space.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

Purpose: The reached set converts the algorithm from a Tree Search into a Graph Search by tracking previously visited states to avoid expanding redundant paths or revisiting cyclic states.

Without reached in DFS: In a grid with cycles or bidirectional movements, DFS without a visited set will follow endless loops (ex: oscillating between (0, 0) and (0, 1)) and get trapped in an infinite recursion/loop, causing the search to fail completely.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your

simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS Optimality: BFS expands nodes layer-by-layer (shallowest first using a FIFO queue). Since step costs are uniform (each move cost = 1), the first time BFS reaches the goal, it is mathematically guaranteed to find the shortest/optimal path.

DFS Suboptimality: DFS explores aggressively down a single branch (deepest first using a LIFO stack). It dives blindly down deep paths until hitting a dead end before backtracking, often finding deep, winding, and highly suboptimal paths to the goal.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS / UCS Memory Bottleneck: Space complexity is $O(b^d)$ (where b is the branching factor and d is the goal depth). BFS must hold all generated nodes in the frontier across the current depth level simultaneously. On a large grid, the exponential memory growth leads to an Out-Of-Memory (OOM) crash very quickly.

DFS Memory Advantage: Space complexity is $O(b \cdot m)$ (where m is maximum search depth). DFS only needs to store the path from the root to the current node along with unexpanded siblings. Its memory requirements scale linearly, consuming dramatically less memory than BFS on massive grids.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING